

SC7 FP300安装及环境配置（Linux）

注意

- 在pcie模式下不需要安装交叉编译工具器
- 大语言模型暂不需要安装tpu-nntc
- 在一个docker容器中即可，不需要多个容器
- 如果在Libsophon中无法ls /dev/bm*查看到

加速器

- 采用标准 PCIE X16 Gen4.0 物理接口，宿主机侧需提供标准的 X16 slot 或至少 X8 in X16 的 PCIE slot（要求尾部开放）。
- 实际应用中建议安装到标准服务器中使用且对卡做相应转速调节保证散热，不建议安装在普通 PC 机/工控机中使用，如需使用，需要外加导风结构并在卡入风口安装高风量、风压风扇。
- 人工智能加速卡 SC7 FP300（全高全长双宽卡）上搭载 8 颗 BM1684X 芯片，可以支持8 x 32 路高清视频硬解码。标准 PCIE 4.0 接口，采用无风扇设计。
- 配置如下图：

```
(llm) ubuntu@ubuntu:~$ free -g
              total        used         free       shared    buff/cache   available
Mem:           125          9            35           0          80          115
Swap:           1           1            0           0           0           0
(11m) ubuntu@ubuntu:~$ sudo cat /proc/version
Linux version 5.15.0-76-generic (build@lcy02-amd64-019) (gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #83~20.04.1
-Ubuntu SMP Wed Jun 21 20:23:31 UTC 2023
```

安装与部署

人工智能加速卡 SC7 FP300 作为标准的 PCIe 卡，请按照如下步骤安装使用。

注意：在安装板卡进入服务器前，请拔掉服务器的主电源和任何网络，少数服务器产品在连接电源时，PCIE 插槽有漏电现象。

1. 使服务器脱离有效运行状态。
2. 关闭服务器电源。
3. 从服务器断开所有电源线，请参阅服务器服务手册。
4. 从机箱卸下箱盖
5. 将板卡插入可用的 PCIe 系统插槽。请轻轻摇晃设备使设备插入到位，不要用力过猛。请勿将 PCIe 板卡安装在 PCI 插槽，反之亦然。PCIe 板卡支持上插，即低位宽的设备可插入高位宽的 PCIe 插槽。
6. 将设备安装支架固定在服务器机箱，根据需要安装托架螺丝，或者啮合服务器固定装置。将卡固定到服务器机箱中
7. 人工智能加速卡 SC7 FP300 需要连接服务器内的 ATX 开关电源，提供 300W 电源
8. 使服务器恢复正常工作，装回箱盖，重新连接电源线以及任何网络电缆，打开系统电源。

设备识别确认

安装完毕后，可进入 Linux 操作系统，查看置于该计算机内的 BM1684X 设备，即对应的加速卡。

```
1 | lspci | grep 168
```

操作系统环境

Ubuntu

如果需要安装 Ubuntu 操作系统，推荐安装以下版本：Ubuntu 16.04、Ubuntu 16.04.5、Ubuntu 16.04.7、Ubuntu 18.04、Ubuntu 18.04.5、Ubuntu 18.04.6、Ubuntu 20.04.4

本机版本

cat /proc/version

Linux version 5.4.0-164-generic (buildd@bos03-amd64-056) (gcc version 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)) #181-Ubuntu SMP Fri Sep 1 13:41:22 UTC 2023

SDK包

具体文件信息参考：[3.1.SDK 简介 — SophonSDKUserGuide v23.07.01 文档\(sophgo.com\)](https://sophon-file.sophon.cn/sophon-product/3.1.SDK简介—SophonSDKUserGuide_v23.07.01文档(sophgo.com))

- 下载SDK包，链接：[https://sophon-file.sophon.cn/sophon-product/3.1.SDK简介—SophonSDKUserGuide_v23.07.01文档\(sophgo.com\)](https://sophon-file.sophon.cn/sophon-product/3.1.SDK简介—SophonSDKUserGuide_v23.07.01文档(sophgo.com))

```
1 sudo apt-get install p7zip
2 sudo apt-get install p7zip-fullcd
3 7z x Release_2370701-public.zip
4 cd Release_2370701-public
```

名称	修改日期	类型	大小
bmmmonkey_20230802_064500	2023/8/28 13:46	文件夹	
bmpanda_20230802_065400	2023/8/28 13:46	文件夹	
libsophon_20230806_182258	2023/8/28 13:46	文件夹	
sophon-demo_20230802_085900	2023/8/28 13:46	文件夹	
sophon-img_20230810_221353	2023/8/28 13:46	文件夹	
sophon-mw_20230807_032400	2023/8/28 13:46	文件夹	
sophon-pipeline_20230802_080800	2023/8/28 13:46	文件夹	
sophon-rpc_20230807_060600	2023/8/28 13:46	文件夹	
sophon-sail_20230802_085400	2023/8/28 13:46	文件夹	
sophon-testhub_20221226_154917	2023/8/28 13:46	文件夹	
tpu-kernel	2023/8/28 13:46	文件夹	
tpu-mlir_20230802_054500	2023/8/28 13:46	文件夹	
tpu-nntc_20230802_054100	2023/8/28 13:46	文件夹	
tpu-perf_v1.2.17	2023/8/28 13:46	文件夹	

Docker安装(前提)

- ```
1 # 安装docker
2 sudo apt-get install docker.io
3 # docker命令免root权限执行
4 # 创建docker用户组，若已有docker组会报错，没关系可忽略
5 sudo groupadd docker
6 # 将当前用户加入docker组
7 sudo gpasswd -a ${USER} docker
8 # 重启docker服务
9 sudo service docker restart
10 # 切换当前会话到新group或重新登录重启X会话
```

```

11 newgrp docker
12
13
14 #添加镜像
15 docker pull sophgo/tpuc_dev:latest
16
17
18 提示：需要logout系统然后重新登录，再使用docker就不需要sudo了。

```

docker环境创建目录

```

1

```

## tpu-mlir环境初始化

- 将压缩包解压到tpu-mlir

```

1 cd tpu-mlir_20230802_054500
2 mkdir tpu-mlir
3 tar zxvf tpu-mlir_v1.2.8-g32d7b3ec-20230802.tar.gz --strip-components=1 -
 C tpu-mlir

```

- 创建docker容器并进入Docker

```

1 cd tpu-mlir
2 # 如果当前系统没有对应的镜像，会自动从docker hub上下载；此处将tpu-mlir的上一级目录
 映射到docker内的/workspace目 录；这里用了8001到8001端口的映射（使用ufw可视化工具
 会用到端口号）。
3 # 如果端口已被占用，请根据实际情况更换为其他未占用的端口。
4 docker run -v $PWD/../../workspace -p 8001:8001 -it
 sophgo/tpuc_dev:latest#成功后自动进入到root模式
5 docker run --privileged myname1234 -v $PWD:/workspace -it
 sophgo/tpuc_dev:latest
6
7
8 docker run --privileged --name newcode -v $PWD:/workspac -p 8001:8001 -
 it sophgo/tpuc_dev:latest
9 '''
10 在Docker容器中运行名为sophgo/tpuc_dev的镜像。同时将主机的当前目录的上一级目录挂
 载到容器的/workspace目录，并将主机的8001端口映射到容器的8001端口，以便通过主机的
 8001端口访问容器内的服务。运行时可以进行交互式操作
11 参数解释
12 docker run : 运行一个容器。
13 -v $PWD/../../workspace : 卷挂载参数，用于将当前目录的上一级目录挂载到容器内
 的/workspace目录。$PWD表示当前工作目录的路径。
14 -p 8001:8001 : 端口映射参数，将主机的8001端口映射到容器的8001端口:可以通过主机
 的8001端口访问容器内的服务。
15 -it: 交互式运行的参数，以便在容器中进行交互式操作。
16 sophgo/tpuc_dev:latest : 这是要运行的Docker镜像的名称和标签。
 sophgo/tpuc_dev是镜像的名称，:latest是标签。
17 '''

```

```

1 '''
2 注：如果编译错误如下
3 docker: Cannot connect to the Docker daemon at
 unix:///var/run/docker.sock. Is the docker daemon running?.
4 See 'docker run --help'.
5 '''
6 sudo service docker status#检查docker状态
7 #可以重启docker服务
8 sudo service docker restart
9 newgrp docker
10 #可以使用 docker version检查，出现Server: 选项正确/使用sudo service docker
 status检查（如下图）

```

```

(11m) ubuntu@ubuntu:~/sc7/Release_230701-public/tpu-mlir_20230802_054500/tpu-mlir$ sudo service docker status
● docker.service - Docker Application Container Engine
 Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
 Active: active (running) since Tue 2023-08-29 09:36:05 CST; 2min 4s ago
 TriggeredBy: ● docker.socket
 Docs: https://docs.docker.com
 Main PID: 2034866 (dockerd)
 Tasks: 43
 Memory: 29.6M
 CGroup: /system.slice/docker.service
 └─2034866 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.184575085+08:00" level=warning msg="Your kernel does not support CPU real-time scheduling"
8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.184589242+08:00" level=warning msg="Your kernel does not support cgroup v1"
8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.184595815+08:00" level=warning msg="Your kernel does not support cgroup v2"
8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.184728088+08:00" level=info msg="Loading containers: start."
8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.275455800+08:00" level=info msg="Default bridge (docker0) is assigned with IP 172.17.0.1. Docker daemon is ready."
8月 29 09:36:05 ubuntu dockerd[2034866]: time="2023-08-29T09:36:05.486120145+08:00" level=info msg="Loading containers: done."

```

这一项是failed表示没有连接

#### • 初始化软件环境

```

1 cd /workspace/tpu-mlir
2 #官方: source scripts/envsetup.sh : 找不到路径
3 source tpu-mlir_v1.2.8-g32d7b3ec-20230802/envsetup.sh
4 #官方: ./build.sh
5 ./tpu-mlir_v1.2.8-g32d7b3ec-20230802/customlayer/build.sh
6

```

```

1 请注意，如果docker stop后重新进入，则需要重新source环境变量。
2
3 在该文件夹下 使用find -name envsetup.sh查找文件位置，根据自身地址修改命令
4 build.sh文件位置如上行一样查找
5 # 使用示例项目代码中自带的模型转换脚本，可以将pytorch模型转为onnx模型。

```

```

root@897528c10e61:/workspace/tpu-mlir# ./tpu-mlir_v1.2.8-g32d7b3ec-20230802/customlayer/build.sh
CMake Warning:
 Ignoring extra path from command line:

 ".."

CMake Error: The source directory "/workspace/tpu-mlir" does not appear to contain CMakeLists.txt.
Specify --help for usage, or press the help button on the CMake GUI.
make: *** No targets specified and no makefile found. Stop.
make: *** No rule to make target 'install'. Stop.
root@897528c10e61:/workspace/tpu-mlir# ls

```

```

1 如遇到上图问题，请进入到build.sh的文件夹下编译
2 即 cd /workspace/tpu-mlir/tpu-mlir_v1.2.8-g32d7b3ec-20230802/customlayer
3 build
4 #出现信息
5 Install the project...
6 -- Install configuration: "Debug"
7 -- Installing: /workspace/tpu-mlir/tpu-mlir_v1.2.8-g32d7b3ec-
 20230802/lib/libbackend_custom.so

```

## tpu-nntc 环境初始化（没用到就不需要）

- 将压缩包解压到tpu-nntc

```
1 cd tpu-nntc_20230802_054100
2 mkdir tpu-nntc
3 tar zxvf tpu-nntc_v3.1.9-62a7181e-230802.tar.gz --strip-components=1 -C
 tpu-nntc
```

- 创建docker容器并进入Docker

```
1 cd tpu-nntc
2 # 如果当前系统没有对应的镜像，会自动从docker hub上下载；此处将tpu-nntc的上一级目录映射到docker内的/workspace目录；这里用了8001到8001端口的映射（使用ufw可视化工具会用到端口号）。
3 # 如果端口已被占用，请根据实际情况更换为其他未占用的端口。
4 docker run -v $PWD/../workspace -p 8001:8001 -it sophgo/tpuc_dev:latest
```

- 初始化软件环境

```
1 cd /workspace/tpu-nntc
2 source scripts/envsetup.sh
3
4 # 请注意，如果docker stop后重新进入，则需要重新source环境变量。
```

## Libsophon环境搭建

具体驱动安装方法，可参考《LIBSOPHON 使用手册》**CHAPTER 3 安装 LIBSOPHON** 中的内容

参考网址：[3. 安装libsophon — LIBSOPHON-GUIDE v23.07.01 文档\(sophgo.com\)](https://sophgo.com/docs/2023/07/01/lib Sophon-GUIDE-v23.07.01/)

准备（不要在一台机器上混用多种安装方式）

- 如果安装了V3.0.0版本及之前版本的SDK驱动，请先卸载旧的驱动

```
1 #进入SDK的安装目录，执行：
2 sudo ./remove_driver_pcie.sh
3 #或者：
4 sudo rmmod bmsophon
5 sudo rm -f /lib/modules/$(uname -r)/kernel/drivers/pci/bmsophon.ko
```

- 查看Linux架构，libsophon包主要文件及功能如下

```
1 uname -m
2 x86_64
3 #如果显示中含有x86_64 :amd64
4 #如果显示中含有aarch64 :arm64
```

- `sophon-driver_0.4.9_`~~amd64~~`.deb` **PCIe加速卡驱动**
- `sophon-libsophon_0.4.9_`~~amd64~~`.deb` **运行时环境（库文件、工具等）**
- `sophon-libsophon-dev_0.4.9_`~~amd64~~`.deb` **开发环境**

只在部署环境上安装，不需要安装开发环境，否则需要安装

- 安装libsophon

```

1 cd libsophon_20230806_182258
2 #安装依赖库，只需要执行一次：
3 sudo apt install dkms libncurses5
4 #安装libsophon:
5 sudo dpkg -i sophon-*_amd64.deb
6 #在终端执行如下命令，或者登出再登入当前用户后即可使用bm-smi等命令：
7 source /etc/profile

```

```

1 #注意：
2 #检查驱动是否安装成功：执行 ls /dev/bm* 看看是否有 /dev/bm-sophonX （X表示0-
 N），如果有表示安装成功。 正常情况下#输出如下信息： /dev/bmdev-ctl /dev/bm-
 sophon0
3 输出： /dev/bmdev-ctl /dev/bm-sophon1 /dev/bm-sophon3 /dev/bm-sophon5
4 /dev/bm-sophon7 /dev/bm-sophon0 /dev/bm-sophon2 /dev/bm-sophon4
 /dev/bm-sophon6
5
6 #安装位置
7 /opt/sophon/
8 └─ driver-0.4.9
9 └─ libsophon-0.4.9
10 | └─ bin
11 | └─ data
12 | └─ include
13 | └─ lib
14 └─ libsophon-current -> /opt/sophon/libsophon-0.4.9
15 #注：若使用不同版本，必须在/opt/sophon下放置，在使用deb包安装
 时，/opt/sophon/libsophon-current会指向最后安装的那个版本。在卸载后，它会指向余下
 的最新版本（如果有的话）
16
17 #错误提示：
18 modprobe: FATAL: Module bmsophon is in use.
19 #解决：手动停止正在使用驱动的程序后，手动执行下面的命令来安装新的deb包里的驱动
20 sudo modprobe -r bmsophon
21 sudo modprobe bmsophon

```

- 如果安装了sophon-mw及sophon-rpc，因为它们对libsophon有依赖关系，请先卸载它们

```

1 sudo apt remove sophon-driver sophon-libsophon
2 #或者：
3 sudo dpkg -r sophon-driver
4 sudo dpkg -r sophon-libsophon-dev
5 sudo dpkg -r sophon-libsophon

```

```

1 #错误解决:
2 #手工卸载驱动:
3 dkms status
4 #检查输出结果, 通常如下:
5 bmsophon, 0.4.9, 5.15.0-41-generic, x86_64: installed
6 #记下前两个字段, 套用到如下命令中:
7 sudo dkms remove -m bmsophon -v 0.4.9 --all
8 #然后再次卸载驱动:
9 sudo apt remove sophon-driver
10 sudo dpkg --purge sophon-driver
11 #彻底清除libsophon:
12 sudo apt purge sophon-libsophon

```

## sophon-mw环境搭建

如果使用Debian/Ubuntu系统 (arm为例) :

|                                                 |                                |
|-------------------------------------------------|--------------------------------|
| sophon-mw-sophon-ffmpeg_v23.07.01_arm64.deb     | 包含ffmpeg/opencv运行时环境 (库文件、工具等) |
| sophon-mw-sophon-ffmpeg-dev_v23.07.01_arm64.deb |                                |
| sophon-mw-sophon-opencv_v23.07.01_arm64.deb     | 开发环境, 在部署环境安装时, 不需要; 否则, 需要安装  |
| sophon-mw-sophon-opencv-dev_v23.07.01_arm64.deb |                                |

### • 安装

```

1 cd sophon-mw_20230807_032400/
2
3 sudo dpkg -i sophon-mw-sophon-ffmpeg_0.7.0_arm64.deb sophon-mw-sophon-
 ffmpeg-dev_0.7.0_arm64.deb
4
5 sudo dpkg -i sophon-mw-sophon-opencv_0.7.0_arm64.deb sophon-mw-sophon-
 opencv-dev_0.7.0_arm64.deb
6
7 #在终端执行如下命令, 或者logout再login当前用户后即可使用安装的工具:
8 source /etc/profile

```

```

1 #注意: 位于SOC模式时, 系统已经预装了:
2 sophon-mw-soc-sophon-ffmpeg
3 sophon-mw-soc-sophon-opencv
4
5 #只需要按照上述步骤安装:
6 sophon-mw-soc-sophon-ffmpeg-dev_v23.07.01_arm64.deb
7 sophon-mw-soc-sophon-opencv-dev_v23.07.01_arm64.deb
8
9 #安装位置:
10 /opt/sophon/
11 ├── libsophon-v23.07.01
12 ├── libsophon-current -> /opt/sophon/libsophon_|ver|
13 ├── sophon-ffmpeg_|ver|
14 | ├── bin
15 | ├── data
16 | ├── include
17 | ├── lib
18 | | ├── cmake
19 | | └── pkgconfig
20 | └── share
21 └── sophon-ffmpeg-latest -> /opt/sophon/sophon-ffmpeg_|ver|

```

```

22 | └─ sophon-opencv_|ver|
23 | └─ bin
24 | └─ data
25 | └─ include
26 | └─ lib
27 | └─ cmake
28 | └─ opencv4
29 | └─ pkgconfig
30 | └─ opencv-python
31 | └─ share
32 | └─ test
33 | └─ sophon-opencv-latest -> /opt/sophon/sophon-opencv_|ver|
34 |

```

- 卸载方式

```

1 | sudo apt remove sophon-mw-sophon-opencv-dev sophon-mw-sophon-opencv
2 | sudo apt remove sophon-mw-sophon-ffmpeg-dev sophon-mw-sophon-ffmpeg
3 | #或者:
4 | sudo dpkg -r sophon-mw-sophon-opencv-dev
5 | sudo dpkg -r sophon-mw-sophon-opencv
6 | sudo dpkg -r sophon-mw-sophon-ffmpeg-dev
7 | sudo dpkg -r sophon-mw-sophon-ffmpeg

```

## 交叉编译工具链

- 安装gcc-aarch64-linux-gnu工具链

```

1 | sudo apt-get install gcc-aarch64-linux-gnu g++-aarch64-linux-gnuls

```

- 解压sophon-img包里的libsophon\_soc\_0.4.9\_aarch64.tar.gz，将lib和include的所有内容拷贝到soc-sdk文件夹。

```

1 | cd sophon-img_20230810_221353
2 | # 创建依赖文件的根目录
3 | mkdir -p soc-sdk
4 | # 解压sophon-img release包里的libsophon_soc_0.4.9_aarch64.tar.gz
5 | tar -zxf libsophon_soc_0.4.9_aarch64.tar.gz
6 | # 将相关的库目录和头文件目录拷贝到依赖文件根目录下
7 | cp -rf libsophon_soc_0.4.9_aarch64/opt/sophon/libsophon-0.4.9/lib ${soc-
 | sdk}
8 | cp -rf libsophon_soc_0.4.9_aarch64/opt/sophon/libsophon-0.4.9/include
 | ${soc-sdk}

```

- 解压sophon-mw包里的sophon-mw-soc<x.y.z>aarch64.tar.gz，将sophon-mw下lib和include的所有内容拷贝到soc-sdk文件夹。

```

1 cd sophon-mw_20230807_032400
2 # 解压sophon-mw包里的sophon-mw-soc_0.7.0_aarch64.tar.gz，其中x.y.z为版本号
3 tar -zxf sophon-mw-soc_0.7.0_aarch64.tar.gz
4 # 将ffmpeg和opencv的库目录和头文件目录拷贝到依赖文件根目录下
5 cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-ffmpeg_0.7.0/lib
 ${soc-sdk}
6 cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-ffmpeg_0.7.0/include
 ${soc-sdk}
7 cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-opencv_0.7.0/lib
 ${soc-sdk}
8 cp -rf sophon-mw-soc_0.7.0_aarch64/opt/sophon/sophon-opencv_0.7.0/include
 ${soc-sdk}

```

- 如果需要使用第三方库，可以使用qemu在x86上构建虚拟环境安装，再将头文件和库文件拷贝到soc-sdk目录中，具体可参考《[sophon-mw使用手册](#)》
- 验证

```

1 which aarch64-linux-gnu-g++
2 # 终端输出内容
3 # /usr/bin/aarch64-linux-gnu-g++

```

```

1 ./model_transform.py --model_name Yolact --model_def
 ./yolact_base_54_800000.onnx --input_shapes [[1,3,550,550]] --mean
 0.0,0.0,0.0 --scale 0.0039216,0.0039216,0.0039216 --keep_aspect_ratio--
 pixel_format rgb --test_input ../COCO2017/0000000000632.jpg --test_result
 yolact_top_outputs.npz --mlir yolact.mlir

```

错误:

```

root@8479c4535253:/workspace/Release_230701-public/libsophon_20230806_182258# sudo apt install dkms libcurses5
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package dkms

```

执行

```

1 sudo apt-get update
2 sudo apt-get upgrade

```

## 交叉编译环境搭建

- 首先安装工具链:

```

sudo apt-get update
sudo apt-get install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu

```

- 解压sophon-img包里的libsophon\_soc\_<x.y.z>\_aarch64.tar.gz，将lib和include的所有内容拷贝到soc-sdk文件夹:

```

cd /workspace/Release_230501-public
创建依赖文件的存放目录
mkdir -p soc-sdk
cd sophon-img_20230605_054900/
解压SDK包里的libsophon_soc_0.4.8_aarch64.tar.gz
tar xzf libsophon_soc_0.4.8_aarch64.tar.gz #解压【重要】
将相关的库目录和头文件目录拷贝到依赖文件存放目录soc-sdk下
cp -rf libsophon_soc_0.4.8_aarch64/opt/sophon/libsophon-0.4.8/lib ../soc-sdk
cp -rf libsophon_soc_0.4.8_aarch64/opt/sophon/libsophon-0.4.8/include ../soc-sdk

```

```
root@61641037ffc1:/workspace# model_transform.py
SOPHGO Toolchain v1.1.0_RC1.137-g90d4b327-20231126
usage: model_transform.py [-h] --model_name MODEL_NAME --model_def MODEL_DEF [--model_data MODEL_DATA]
 [--input_shapes INPUT_SHAPES] [--input_types INPUT_TYPES] [--output_names OUTPUT_NAMES]
 [--test_input TEST_INPUT] [--test_result TEST_RESULT] [--tolerance TOLERANCE] [--excepts EXCEPTS]
 [--add_postprocess {,yolov3,yolov5,yolov8,ssd,bnr}] [--onnx_sim {,skip_fuse_bn}] [--debug] [--mlir MLIR]
 [--resize_dims RESIZE_DIMS] [--keep_aspect_ratio] [--keep_ratio_mode {letterbox,short_side_scale}]
 [--mean [MEAN]] [--scale SCALE] [--pixel_format {rgb,bgr,gray,rgba,gbgr,grbg,bggr,rggb}]
 [--channel_format {nhwc,nchw,none}] [--pad_value PAD_VALUE] [--pad_type {normal,center}]
 [--white_level WHITE_LEVEL] [--black_level BLACK_LEVEL] [--preprocess_list PREPROCESS_LIST]
 [--debug_cmd DEBUG_CMD]
 [unknown_params]

model_transform.py: error: the following arguments are required: --model_name, --model_def, --mlir
root@61641037ffc1:/workspace#
```